```
import { sys } from 'cc';

/**
 * 广告管理器 — 统一管理各平台广告播放。
 * 目前支持微信小游戏(wx)与抖音小游戏(tt)的激励视频广告。
 */
export enum AdPlatform {
    Auto = 'auto',
    WeChat = 'wechat',
    Douyin = 'douyin',
}

export enum AdPlayResult {
    Completed = 'completed',
    Skipped = 'skipped',
    Failed = 'failed',
    Unsupported = 'unsupported',
    Busy = 'busy',
}

export interface RewardedVideoAdOptions {
    adUnitId?: string;
    platform?: AdPlatform;
}

export interface RewardedVideoAdResult {
    result: AdPlayResult;
    platform: AdPlatform;
    isEnded: boolean;
    message?: string;
    err?: unknown;
}

type MiniGameAdCloseResult = {
    isEnded?: boolean;
};

type MiniGameRewardedVideoAd = {
    show: () => Promise<void> | void;
    load?: () => Promise<void> | void;
    destroy?: () => void;
    onLoad?: (callback: () => void) => void;
    offLoad?: (callback: () => void) => void;
    onClose?: (callback: (res?: MiniGameAdCloseResult) => void) => void;
    offClose?: (callback: (res?: MiniGameAdCloseResult) => void) => void;
    onError?: (callback: (err: unknown) => void) => void;
    offError?: (callback: (err: unknown) => void) => void;
};

type MiniGameAdRuntime = {
```

```
    createRewardedVideoAd?: (options: { adUnitId: string }) => MiniGameRewardedVideoAd;
};

export class AdManager {
    private static _instance: AdManager;

    private _rewardedAdUnitIds: Map<AdPlatform, string> = new Map();
    private _rewardedAdCache: Map<string, MiniGameRewardedVideoAd> = new Map();
    private _isPlayingRewardedVideo: boolean = false;
    private _initialized: boolean = false;
    private _currentPlatform: AdPlatform = AdPlatform.Auto;
    private _currentRuntime: MiniGameAdRuntime | null = null;

    static getInstance(): AdManager {
        if (!this._instance) {
            this._instance = new AdManager();
        }
        return this._instance;
    }

    init(): void {
        if (this._initialized) return;

        this._currentPlatform = this.resolvePlatformBySystem();
        this._currentRuntime = this.getRuntime(this._currentPlatform);
        this._initialized = true;

       if (this._currentPlatform === AdPlatform.Auto || !this._currentRuntime?.createRewardedVideoAd) {
            console.warn('AdManager: 当前平台暂不支持广告');
        }
    }

    setRewardedVideoAdUnitId(platform: AdPlatform, adUnitId: string): void {
        if (!adUnitId) {
            this._rewardedAdUnitIds.delete(platform);
            return;
        }
        this._rewardedAdUnitIds.set(platform, adUnitId);
    }

    setRewardedVideoAdUnitIds(adUnitIds: Partial<Record<AdPlatform, string>>): void {
        Object.keys(adUnitIds).forEach(key => {
            const platform = key as AdPlatform;
            if (platform === AdPlatform.Auto) return;
            this.setRewardedVideoAdUnitId(platform, adUnitIds[platform]);
        });
    }

    async playRewardedVideoAd(options: RewardedVideoAdOptions = {}): Promise<RewardedVideoAdResult> {
        if (this._isPlayingRewardedVideo) {
```

```
    return {
        result: AdPlayResult.Busy,
        platform: options.platform ?? AdPlatform.Auto,
        isEnded: false,
        message: '已有激励视频广告正在播放',
    };
}

if (!this._initialized) {
    this.init();
}

const platform = this.resolvePlatform(options.platform);
if (platform === AdPlatform.Auto) {
    return {
        result: AdPlayResult.Unsupported,
        platform,
        isEnded: false,
        message: '当前运行环境不支持激励视频广告',
    };
}

const runtime = this.getRuntimeByPlatform(platform);
if (!runtime?.createRewardedVideoAd) {
    return {
        result: AdPlayResult.Unsupported,
        platform,
        isEnded: false,
        message: `当前平台未提供激励视频广告接口 [${platform}]`,
    };
}

const adUnitId = options.adUnitId || this._rewardedAdUnitIds.get(platform);
if (!adUnitId) {
    return {
        result: AdPlayResult.Failed,
        platform,
        isEnded: false,
        message: `未配置激励视频广告位 ID [${platform}]`,
    };
}

this._isPlayingRewardedVideo = true;
try {
    const ad = this.getOrCreateRewardedVideoAd(platform, adUnitId, runtime);
    return await this.showRewardedVideoAd(ad, platform);
} catch (err) {
    return {
        result: AdPlayResult.Failed,
        platform,
```

```
            isEnded: false,
            message: '激励视频广告播放失败',
            err,
        };
    } finally {
        this._isPlayingRewardedVideo = false;
    }
}

destroy(): void {
    this._rewardedAdCache.forEach(ad => ad.destroy?.());
    this._rewardedAdCache.clear();
    this._rewardedAdUnitIds.clear();
    this._isPlayingRewardedVideo = false;
    this._initialized = false;
    this._currentPlatform = AdPlatform.Auto;
    this._currentRuntime = null;
}

private resolvePlatform(platform: AdPlatform = AdPlatform.Auto): AdPlatform {
    if (platform !== AdPlatform.Auto) return platform;
    if (this._currentPlatform !== AdPlatform.Auto) return this._currentPlatform;

    return this.resolvePlatformBySystem();
}

private resolvePlatformBySystem(): AdPlatform {
    if (sys.platform === sys.Platform.WECHAT_GAME) return AdPlatform.WeChat;
    if (sys.platform === sys.Platform.BYTEDANCE_MINI_GAME) return AdPlatform.Douyin;

    return AdPlatform.Auto;
}

private getRuntimeByPlatform(platform: AdPlatform): MiniGameAdRuntime | null {
    if (platform === this._currentPlatform) return this._currentRuntime;
    return this.getRuntime(platform);
}

private getRuntime(platform: AdPlatform): MiniGameAdRuntime | null {
    const globalObj = globalThis as unknown as { wx?: MiniGameAdRuntime; tt?: MiniGameAdRuntime };
    if (platform === AdPlatform.WeChat) return globalObj.wx ?? null;
    if (platform === AdPlatform.Douyin) return globalObj.tt ?? null;
    return null;
}

private getOrCreateRewardedVideoAd(platform: AdPlatform, adUnitId: string, runtime: MiniGameAdRuntime): MiniGameRewardedVideoAd {
    const cacheKey = `${platform}:${adUnitId}`;
    const cachedAd = this._rewardedAdCache.get(cacheKey);
    if (cachedAd) return cachedAd;
```

```
    const ad = runtime.createRewardedVideoAd({ adUnitId });
    this._rewardedAdCache.set(cacheKey, ad);
    return ad;
}

private showRewardedVideoAd(ad: MiniGameRewardedVideoAd, platform: AdPlatform): Promise<RewardedVideoAdResult> {
    return new Promise(resolve => {
        let finished = false;

        const cleanup = (): void => {
            ad.offClose?.(onClose);
            ad.offError?.(onError);
        };

        const finish = (result: RewardedVideoAdResult): void => {
            if (finished) return;
            finished = true;
            cleanup();
            resolve(result);
        };

        const onClose = (res?: MiniGameAdCloseResult): void => {
            const isEnded = res?.isEnded !== false;
            finish({
                result: isEnded ? AdPlayResult.Completed : AdPlayResult.Skipped,
                platform,
                isEnded,
                message: isEnded ? '激励视频广告播放完成' : '用户提前关闭激励视频广告',
            });
        };

        const onError = (err: unknown): void => {
            finish({
                result: AdPlayResult.Failed,
                platform,
                isEnded: false,
                message: '激励视频广告发生错误',
                err,
            });
        };

        ad.onClose?.(onClose);
        ad.onError?.(onError);

        Promise.resolve(ad.show()).catch(() => {
            Promise.resolve(ad.load?.())
                .then(() => Promise.resolve(ad.show()))
                .catch(onError);
        });
    });
```

```
    }
}
import { AudioClip, AudioSource, Node, resources } from 'cc';

/**
 * 音频管理器 — 管理背景音乐和音效的播放
 * 引擎层：依赖 Cocos Creator 音频系统
 */
export class AudioManager {
    private static _instance: AudioManager;

    private _bgmSource: AudioSource = null;
    private _sfxSource: AudioSource = null;
    private _bgmVolume: number = 1.0;
    private _sfxVolume: number = 1.0;
    private _bgmMuted: boolean = false;
    private _sfxMuted: boolean = false;
    private _audioNode: Node = null;

    /** 音效缓存 */
    private _clipCache: Map<string, AudioClip> = new Map();

    static getInstance(): AudioManager {
        if (!this._instance) {
            this._instance = new AudioManager();
        }
        return this._instance;
    }

    /**
     * 初始化，需要传入一个常驻节点
     * AudioSource 组件会挂载到该节点上
     */
    init(persistNode: Node): void {
        if (this._audioNode) return;

        this._audioNode = new Node('AudioManager');
        this._audioNode.parent = persistNode;

        this._bgmSource = this._audioNode.addComponent(AudioSource);
        this._bgmSource.loop = true;
        this._bgmSource.playOnAwake = false;

        this._sfxSource = this._audioNode.addComponent(AudioSource);
        this._sfxSource.loop = false;
        this._sfxSource.playOnAwake = false;
    }

    // ==================== BGM ====================
```

```
playBGM(path: string): void {
    this.loadClip(path, (clip) => {
        if (!this._bgmSource) return;
        this._bgmSource.stop();
        this._bgmSource.clip = clip;
        this._bgmSource.volume = this._bgmMuted ? 0 : this._bgmVolume;
        this._bgmSource.play();
    });
}

stopBGM(): void {
    this._bgmSource?.stop();
}

pauseBGM(): void {
    this._bgmSource?.pause();
}

resumeBGM(): void {
    if (this._bgmSource && !this._bgmSource.playing) {
        this._bgmSource.play();
    }
}

setBGMVolume(vol: number): void {
    this._bgmVolume = Math.max(0, Math.min(1, vol));
    if (this._bgmSource && !this._bgmMuted) {
        this._bgmSource.volume = this._bgmVolume;
    }
}

setBGMMuted(muted: boolean): void {
    this._bgmMuted = muted;
    if (this._bgmSource) {
        this._bgmSource.volume = muted ? 0 : this._bgmVolume;
    }
}

get isBGMMuted(): boolean { return this._bgmMuted; }

// ==================== SFX ====================

playSFX(path: string): void {
    if (this._sfxMuted) return;
    this.loadClip(path, (clip) => {
        if (!this._sfxSource) return;
        this._sfxSource.playOneShot(clip, this._sfxVolume);
    });
}
```

```
    setSFXVolume(vol: number): void {
        this._sfxVolume = Math.max(0, Math.min(1, vol));
    }

    setSFXMuted(muted: boolean): void {
        this._sfxMuted = muted;
    }

    get isSFXMuted(): boolean { return this._sfxMuted; }

    // ==================== 内部 ====================

    private loadClip(path: string, callback: (clip: AudioClip) => void): void {
        const cached = this._clipCache.get(path);
        if (cached) {
            callback(cached);
            return;
        }

        resources.load(path, AudioClip, (err, clip) => {
            if (err) {
                console.warn(`AudioManager: 加载音频失败 [${path}]`, err);
                return;
            }
            this._clipCache.set(path, clip);
            callback(clip);
        });
    }

    clearCache(): void {
        this._clipCache.clear();
    }

    destroy(): void {
        this.stopBGM();
        this._clipCache.clear();
        if (this._audioNode) {
            this._audioNode.destroy();
            this._audioNode = null;
        }
        this._bgmSource = null;
        this._sfxSource = null;
    }
}
import { FlatBuffersRuntime, ByteBuffer } from './FlatBuffersRuntime';

/**
 * 配置表访问器接口 — 由 FlatBuffers 生成代码实现
 * 每个配置表对应一个访问器，提供类型安全的数据读取
 */
```

```
export interface IConfigAccessor<T> {
    /** 获取记录总数 */
    getCount(): number;
    /** 按索引获取记录 */
    getByIndex(index: number): T | null;
    /** 按主键（ID）查询单条记录 */
    getById(id: number | string): T | null;
    /** 遍历所有记录 */
    forEach(callback: (item: T, index: number) => void): void;
    /** 获取所有记录 */
    getAll(): T[];
}

/**
 * 配置管理器 — 引擎层，统一管理所有配置表的访问器
 * 作为基础模块供所有游戏模块复用
 *
 * 使用方式:
 *   ConfigManager.getInstance().registerAccessor('enemy', new EnemyConfigAccessor(buffer));
 *   const enemy = ConfigManager.getInstance().getById<EnemyConfig>('enemy', 1001);
 */
export class ConfigManager {
    private static _instance: ConfigManager;

    /** 表名 → 配置访问器 */
    private _accessors: Map<string, IConfigAccessor<any>> = new Map();

    static getInstance(): ConfigManager {
        if (!this._instance) {
            this._instance = new ConfigManager();
        }
        return this._instance;
    }

    /**
     * 注册配置访问器
     * @param tableName 配置表名（与 FlatBuffersRuntime 中的表名一致）
     * @param accessor  类型安全的配置访问器实例
     */
    registerAccessor<T>(tableName: string, accessor: IConfigAccessor<T>): void {
        if (this._accessors.has(tableName)) {
            console.warn(`ConfigManager: 访问器 [${tableName}] 已存在，将被覆盖`);
        }
        this._accessors.set(tableName, accessor);
    }

    /**
     * 获取配置访问器
     * @param tableName 配置表名
     * @returns 访问器实例，未注册则返回 null
```

```
 */
getAccessor<T>(tableName: string): IConfigAccessor<T> | null {
    const accessor = this._accessors.get(tableName);
    if (!accessor) {
        console.warn(`ConfigManager: 未注册的访问器 [${tableName}]`);
        return null;
    }
    return accessor as IConfigAccessor<T>;
}

/**
 * 按主键查询单条记录（便捷方法）
 * @param tableName 配置表名
 * @param id        主键值
 * @returns 记录数据，未找到则返回 null
 */
getById<T>(tableName: string, id: number | string): T | null {
    const accessor = this.getAccessor<T>(tableName);
    if (!accessor) return null;
    return accessor.getById(id);
}

/**
 * 获取指定表的所有记录（便捷方法）
 * @param tableName 配置表名
 * @returns 所有记录数组，未注册则返回空数组
 */
getAll<T>(tableName: string): T[] {
    const accessor = this.getAccessor<T>(tableName);
    if (!accessor) return [];
    return accessor.getAll();
}

/**
 * 获取指定表的记录总数
 * @param tableName 配置表名
 * @returns 记录数，未注册则返回 0
 */
getCount(tableName: string): number {
    const accessor = this._accessors.get(tableName);
    if (!accessor) return 0;
    return accessor.getCount();
}

/** 检查指定表的访问器是否已注册 */
hasAccessor(tableName: string): boolean {
    return this._accessors.has(tableName);
}

/** 获取所有已注册的表名 */
```

```
    getRegisteredTables(): string[] {
        return Array.from(this._accessors.keys());
    }

    /** 移除指定表的访问器 */
    removeAccessor(tableName: string): void {
        this._accessors.delete(tableName);
    }

    /** 清空所有访问器 */
    clear(): void {
        this._accessors.clear();
    }
}
import { BufferAsset, resources } from 'cc';

/**
 * 轻量级 ByteBuffer 封装 — 提供零拷贝读取能力
 * 包装 ArrayBuffer + DataView，供 FlatBuffers 生成代码使用
 */
export class ByteBuffer {
    private _bytes: Uint8Array;
    private _view: DataView;
    private _position: number = 0;

    constructor(bytes: Uint8Array) {
        this._bytes = bytes;
        this._view = new DataView(bytes.buffer, bytes.byteOffset, bytes.byteLength);
    }

    /** 从 ArrayBuffer 创建 */
    static fromArrayBuffer(buffer: ArrayBuffer): ByteBuffer {
        return new ByteBuffer(new Uint8Array(buffer));
    }

    /** 获取底层字节数组（零拷贝） */
    bytes(): Uint8Array {
        return this._bytes;
    }

    /** 获取 DataView */
    dataView(): DataView {
        return this._view;
    }

    /** 获取缓冲区总长度 */
    capacity(): number {
        return this._bytes.byteLength;
    }
```

```
    /** 获取/设置当前读取位置 */
    position(): number {
        return this._position;
    }

    setPosition(pos: number): void {
        this._position = pos;
    }

    /** 读取 int32 */
    readInt32(offset: number): number {
        return this._view.getInt32(offset, true);
    }

    /** 读取 float32 */
    readFloat32(offset: number): number {
        return this._view.getFloat32(offset, true);
    }

    /** 读取 uint8 (bool) */
    readUint8(offset: number): number {
        return this._view.getUint8(offset);
    }

    /** 读取 int16 */
    readInt16(offset: number): number {
        return this._view.getInt16(offset, true);
    }

    /** 读取 uint16 */
    readUint16(offset: number): number {
        return this._view.getUint16(offset, true);
    }
}

/**
 * FlatBuffers 配置加载错误码
 */
export enum ConfigLoadError {
    None = 0,
    FileNotFound = 1,
    ParseFailed = 2,
    InvalidFormat = 3,
}

/**
 * 配置加载结果
 */
export interface ConfigLoadResult {
    success: boolean;
```

```
    error: ConfigLoadError;
    message: string;
}

/**
 * FlatBuffers 运行时 — 引擎层，管理二进制配置文件的加载和零拷贝读取
 * 作为基础模块供所有游戏模块复用
 *
 * 使用方式:
 *   await FlatBuffersRuntime.getInstance().loadAll({ enemy: 'config/enemy', weapon: 'config/weapon' });
 *   const buf = FlatBuffersRuntime.getInstance().getBuffer('enemy');
 */
export class FlatBuffersRuntime {
    private static _instance: FlatBuffersRuntime;

    /** 已加载的二进制配置缓存：表名 → ByteBuffer */
    private _buffers: Map<string, ByteBuffer> = new Map();

    static getInstance(): FlatBuffersRuntime {
        if (!this._instance) {
            this._instance = new FlatBuffersRuntime();
        }
        return this._instance;
    }

    /**
     * 异步加载单个 Binary_Config 文件
     * @param tableName 配置表名（用于缓存键）
     * @param path      资源路径（resources 目录下的相对路径）
     * @returns 加载结果
     */
    loadConfig(tableName: string, path: string): Promise<ConfigLoadResult> {
        return new Promise((resolve) => {
            resources.load(path, BufferAsset, (err, asset) => {
                if (err || !asset) {
                const message = `FlatBuffersRuntime: 加载配置失败 [${tableName}] path=${path} ${err?.message ?? ''}`;
                    console.error(message);
                    resolve({
                        success: false,
                        error: ConfigLoadError.FileNotFound,
                        message,
                    });
                    return;
                }

                try {
                    const arrayBuffer = asset.buffer();
                    if (!arrayBuffer || arrayBuffer.byteLength === 0) {
                        const message = `FlatBuffersRuntime: 配置数据为空 [${tableName}]`;
                        console.error(message);
```

```
                    resolve({
                        success: false,
                        error: ConfigLoadError.InvalidFormat,
                        message,
                    });
                    return;
                }

                // 零拷贝：直接包装 ArrayBuffer，不做额外拷贝
                const buffer = ByteBuffer.fromArrayBuffer(arrayBuffer);
                this._buffers.set(tableName, buffer);

                resolve({
                    success: true,
                    error: ConfigLoadError.None,
                    message: '',
                });
            } catch (e) {
              const message = `FlatBuffersRuntime: 解析配置失败 [${tableName}] ${(e as Error).message}`;
                console.error(message);
                resolve({
                    success: false,
                    error: ConfigLoadError.ParseFailed,
                    message,
                });
            }
        });
    });
}

/**
 * 批量加载所有配置（并行）
 * @param configMap 表名 → 资源路径 的映射
 * @returns 所有加载结果
 */
async loadAll(configMap: Record<string, string>): Promise<Map<string, ConfigLoadResult>> {
    const results = new Map<string, ConfigLoadResult>();
    const entries = Object.entries(configMap);

    const promises = entries.map(async ([tableName, path]) => {
        const result = await this.loadConfig(tableName, path);
        results.set(tableName, result);
    });

    await Promise.all(promises);
    return results;
}

/**
 * 获取指定表的 ByteBuffer（零拷贝读取）
```

```
     * @param tableName 配置表名
     * @returns ByteBuffer 实例，未加载则返回 null
     */
    getBuffer(tableName: string): ByteBuffer | null {
        return this._buffers.get(tableName) ?? null;
    }

    /** 检查指定表是否已加载 */
    hasBuffer(tableName: string): boolean {
        return this._buffers.has(tableName);
    }

    /** 获取所有已加载的表名 */
    getLoadedTables(): string[] {
        return Array.from(this._buffers.keys());
    }

    /** 释放指定表的缓存 */
    releaseConfig(tableName: string): void {
        this._buffers.delete(tableName);
    }

    /** 释放所有缓存 */
    releaseAll(): void {
        this._buffers.clear();
    }
}
import { _decorator, Node } from "cc";
import { UIManager } from "./ui/UIManager";
import { AudioManager } from "./AudioManager";
import { ResManager } from "./ResManager";
import { NodePoolManager } from "./NodePool";
import { ConfigManager } from "./ConfigManager";
import { FlatBuffersRuntime } from "./FlatBuffersRuntime";
import { AdManager, AdPlatform } from "./AdManager";
import { TimerManager } from "../framework/TimerManager";
import { StorageManager } from "../framework/StorageManager";
import { PoolManager } from "../framework/ObjectPool";
import { EventManager } from "../framework/EventManager";
import { FrameworkEvent } from "../framework/FrameworkEvent";

const { ccclass } = _decorator;

/**
 * 游戏管理器 — 引擎层，统一初始化和驱动所有子系统
 * 桥接框架层（纯逻辑）和引擎层（Cocos 依赖），
 * 业务初始化通过 onGameReady 回调注入
 */
@ccclass('GameManager')
export class GameManager {
```

```
private static m_Instance: GameManager = null;

static GetInstance(): GameManager {
    if (this.m_Instance == null) {
        this.m_Instance = new GameManager();
    }
    return this.m_Instance;
}

private m_GameWorldRoot: Node = null;
private m_Initialized: boolean = false;

/**
 * 初始化所有子系统
 * @param gameWorldRoot 游戏世界根节点
 * @param uiRoot UI 根节点
 * @param persistNode 常驻节点（用于挂载 AudioSource 等）
 * @param onGameReady 游戏层初始化回调（注册 UI、加载首屏等，可异步加载配置）
 */
Init(
    gameWorldRoot: Node,
    uiRoot: Node,
    persistNode: Node,
    onGameReady?: () => void | Promise<void>
): void {
    if (this.m_Initialized) return;
    this.m_Initialized = true;

    this.m_GameWorldRoot = gameWorldRoot;

    // 1. 框架层 — 存储
    StorageManager.getInstance();

    // 2. 引擎层 — 音频
    AudioManager.getInstance().init(persistNode);

    // 3. 引擎层 — 广告
    AdManager.getInstance().init();
    // AdManager.getInstance().setRewardedVideoAdUnitId(AdPlatform.WeChat, "微信广告位 ID");
    // AdManager.getInstance().setRewardedVideoAdUnitId(AdPlatform.Douyin, "抖音广告位 ID");

    // 4. 引擎层 — UI
    UIManager.GetInstance().Init(uiRoot);

    // 5. 游戏层初始化（注册 UI 面板、设置存储前缀、打开首屏等）
    if (onGameReady) {
        try {
            Promise.resolve(onGameReady()).catch((err) => {
                console.error("GameManager: 游戏层初始化失败", err);
            });
```

```
            } catch (err) {
                console.error("GameManager: 游戏层初始化失败", err);
            }
        }
    }

    /** 每帧更新，由 Launcher 调用 */
    Update(dt: number): void {
        TimerManager.getInstance().update(dt);
    }

    /** 每帧 LateUpdate，由 Launcher 调用 */
    LateUpdate(_dt: number): void {
        // 预留
    }

    /** 销毁所有子系统 */
    Destroy(): void {
        TimerManager.getInstance().clear();
        PoolManager.getInstance().clearAll();
        NodePoolManager.getInstance().clearAll();
        AudioManager.getInstance().destroy();
        UIManager.GetInstance().Destroy();
        EventManager.getInstance().clear();
        ConfigManager.getInstance().clear();
        FlatBuffersRuntime.getInstance().releaseAll();
        AdManager.getInstance().destroy();
        ResManager.getInstance().releaseAll();
        this.m_Initialized = false;
    }

    /** 游戏进入后台 */
    PauseGame(): void {
        AudioManager.getInstance().pauseBGM();
        TimerManager.getInstance().pauseAll();
        EventManager.getInstance().emit(FrameworkEvent.GamePaused);
    }

    /** 游戏回到前台 */
    ResumeGame(): void {
        AudioManager.getInstance().resumeBGM();
        TimerManager.getInstance().resumeAll();
        EventManager.getInstance().emit(FrameworkEvent.GameResumed);
    }

    /** 获取游戏世界根节点 */
    getGameWorldRoot(): Node {
        return this.m_GameWorldRoot;
    }
}
```

```
import { _decorator, Camera, Component, director, Node, game, Game } from 'cc';
import { ScreenAdapter } from "./ScreenAdapter";
import { GameManager } from "./GameManager";
import { initCommonGame } from "../Game/CommonGame/CommonGameEntry";
import { PlatformManager } from "./PlatformManager";

const { ccclass, property } = _decorator;

/**
 * 游戏入口 — 常驻节点，驱动 GameManager 生命周期
 * 引擎层：唯一与游戏层耦合的地方是 onGameReady 回调
 * 切换玩法时只需替换 initCommonGame 为其他游戏的入口函数
 */
@ccclass('Launcher')
export class Launcher extends Component {
    @property(ScreenAdapter)
    public m_ScreenAdapter: ScreenAdapter = null;
    @property(Camera)
    m_Camera: Camera = null;
    @property(Node)
    m_UIRoot: Node = null;
    @property(Node)
    m_GameWorld: Node = null;

    protected onLoad(): void {
        director.addPersistRootNode(this.node);
        PlatformManager.getInstance().init();

        game.on(Game.EVENT_HIDE, this.onGameHide, this);
        game.on(Game.EVENT_SHOW, this.onGameShow, this);

        GameManager.GetInstance().Init(
            this.m_GameWorld,
            this.m_UIRoot,
            this.node,
            initCommonGame
        );
    }

    protected update(dt: number): void {
        GameManager.GetInstance().Update(dt);
    }

    protected lateUpdate(dt: number): void {
        GameManager.GetInstance().LateUpdate(dt);
    }

    protected onDestroy(): void {
        game.off(Game.EVENT_HIDE, this.onGameHide, this);
        game.off(Game.EVENT_SHOW, this.onGameShow, this);
```

```
        GameManager.GetInstance().Destroy();
    }

    private onGameHide(): void {
        GameManager.GetInstance().PauseGame();
    }

    private onGameShow(): void {
        GameManager.GetInstance().ResumeGame();
    }
}
import { instantiate, Node, Prefab } from 'cc';

/**
 * Cocos Node 对象池 — 减少频繁 instantiate/destroy 的开销
 * 引擎层：依赖 Cocos Creator 节点系统
 *
 * 使用方式:
 *   const pool = new NodePool(bulletPrefab, 20);
 *   const bullet = pool.get();
 *   bullet.parent = this.node;
 *   pool.put(bullet);
 */
export class NodePool {
    private _prefab: Prefab;
    private _pool: Node[] = [];
    private _maxSize: number;

    constructor(prefab: Prefab, initSize: number = 0, maxSize: number = 100) {
        this._prefab = prefab;
        this._maxSize = maxSize;

        for (let i = 0; i < initSize; i++) {
            const node = instantiate(this._prefab);
            node.active = false;
            this._pool.push(node);
        }
    }

    /** 从池中获取一个节点，池空则新建 */
    get(): Node {
        let node: Node;
        while (this._pool.length > 0) {
            node = this._pool.pop()!;
            if (node && node.isValid) {
                node.active = true;
                return node;
            }
        }
```

```
        node = instantiate(this._prefab);
        node.active = true;
        return node;
    }

    /** 回收节点到池中 */
    put(node: Node): void {
        if (!node || !node.isValid) return;

        node.removeFromParent();
        node.active = false;

        if (this._pool.includes(node)) return;

        if (this._pool.length < this._maxSize) {
            this._pool.push(node);
        } else {
            node.destroy();
        }
    }

    /** 当前池中空闲节点数 */
    get size(): number {
        return this._pool.length;
    }

    /** 清空并销毁池中所有节点 */
    clear(): void {
        this._pool.forEach(node => {
            if (node && node.isValid) {
                node.destroy();
            }
        });
        this._pool.length = 0;
    }
}

/**
 * Node 对象池管理器 — 按名称管理多个 Node 对象池
 */
export class NodePoolManager {
    private static _instance: NodePoolManager;
    private _pools: Map<string, NodePool> = new Map();

    static getInstance(): NodePoolManager {
        if (!this._instance) {
            this._instance = new NodePoolManager();
        }
        return this._instance;
    }
```

```
    /** 创建一个命名对象池 */
    createPool(name: string, prefab: Prefab, initSize: number = 0, maxSize: number = 100): NodePool {
        if (this._pools.has(name)) {
            console.warn(`NodePoolManager: 池 [${name}] 已存在，将被覆盖`);
            this._pools.get(name)!.clear();
        }
        const pool = new NodePool(prefab, initSize, maxSize);
        this._pools.set(name, pool);
        return pool;
    }

    /** 从命名池获取节点 */
    get(name: string): Node | null {
        const pool = this._pools.get(name);
        if (!pool) {
            console.warn(`NodePoolManager: 池 [${name}] 不存在`);
            return null;
        }
        return pool.get();
    }

    /** 回收节点到命名池 */
    put(name: string, node: Node): void {
        const pool = this._pools.get(name);
        if (!pool) {
            console.warn(`NodePoolManager: 池 [${name}] 不存在，节点将被销毁`);
            if (node && node.isValid) {
                node.destroy();
            }
            return;
        }
        pool.put(node);
    }

    /** 清空指定池 */
    clearPool(name: string): void {
        this._pools.get(name)?.clear();
        this._pools.delete(name);
    }

    /** 清空所有池 */
    clearAll(): void {
        this._pools.forEach(pool => pool.clear());
        this._pools.clear();
    }
}
import { sys } from 'cc';

export enum MiniGamePlatform {
```

```
    Auto = 'auto',
    WeChat = 'wechat',
    Douyin = 'douyin',
}

export enum PlatformResult {
    Success = 'success',
    Failed = 'failed',
    Unsupported = 'unsupported',
}

export interface PlatformLoginResult {
    result: PlatformResult;
    platform: MiniGamePlatform;
    code?: string;
    userInfo?: unknown;
    message?: string;
    err?: unknown;
}

export interface PlatformRankResult {
    result: PlatformResult;
    platform: MiniGamePlatform;
    message?: string;
    err?: unknown;
}

export interface PlatformShareOptions {
    title?: string;
    imageUrl?: string;
    query?: string;
}

export interface PlatformShareResult {
    result: PlatformResult;
    platform: MiniGamePlatform;
    message?: string;
    err?: unknown;
}

export interface PlatformPrivacySettingResult {
    result: PlatformResult;
    platform: MiniGamePlatform;
    needAuthorization?: boolean;
    privacyContractName?: string;
    message?: string;
    err?: unknown;
}

export interface PlatformPrivacyAuthorizeResult {
```

```
    result: PlatformResult;
    platform: MiniGamePlatform;
    message?: string;
    err?: unknown;
}

export interface PlatformSidebarResult {
    result: PlatformResult;
    platform: MiniGamePlatform;
    isExist?: boolean;
    isFromSidebar?: boolean;
    message?: string;
    err?: unknown;
}

export interface PlatformGameClubResult {
    result: PlatformResult;
    platform: MiniGamePlatform;
    message?: string;
    err?: unknown;
}

export interface PlatformRankUserData {
    nickname: string;
    avatarUrl?: string;
    score: number;
    isSelf?: boolean;
}

export interface PlatformRankListResult extends PlatformRankResult {
    data?: PlatformRankUserData[];
    self?: PlatformRankUserData;
}

export interface PlatformUserCloudStorageItem {
    key: string;
    value: string;
}

type MiniGameLoginResult = {
    code?: string;
};

type MiniGameUserInfoResult = {
    userInfo?: unknown;
};

type MiniGamePrivacySettingResult = {
    needAuthorization?: boolean;
    privacyContractName?: string;
```

```
};

type MiniGameSceneCheckResult = {
    isExist?: boolean;
};

type MiniGameShowOptions = {
    scene?: string;
    launch_from?: string;
    location?: string;
    query?: Record<string, string>;
    refererInfo?: unknown;
};

type MiniGameOpenDataContext = {
    canvas?: unknown;
    postMessage?: (message: unknown) => void;
};

type MiniGamePageManager = {
    load?: (options: { openlink: string }) => Promise<unknown>;
    show?: () => void;
    hide?: () => void;
    destroy?: () => void;
    [key: string]: unknown;
};

type MiniGameCloudStorageData = {
    key: string;
    value: string;
};

type MiniGameFriendCloudStorageItem = {
    nickname?: string;
    avatarUrl?: string;
    KVDataList?: MiniGameCloudStorageData[];
};

type MiniGameRuntime = {
    login?: (options: {
        success?: (res: MiniGameLoginResult) => void;
        fail?: (err: unknown) => void;
    }) => void;
    getUserInfo?: (options: {
        success?: (res: MiniGameUserInfoResult) => void;
        fail?: (err: unknown) => void;
    }) => void;
    setUserCloudStorage?: (options: {
        KVDataList: PlatformUserCloudStorageItem[];
        success?: () => void;
```

```
        fail?: (err: unknown) => void;
    }) => void;
    getUserCloudStorage?: (options: {
        keyList: string[];
        success?: (res: { KVDataList?: MiniGameCloudStorageData[] }) => void;
        fail?: (err: unknown) => void;
    }) => void;
    getFriendCloudStorage?: (options: {
        keyList: string[];
        success?: (res: { data?: MiniGameFriendCloudStorageItem[] }) => void;
        fail?: (err: unknown) => void;
    }) => void;
    shareAppMessage?: (options: {
        title?: string;
        imageUrl?: string;
        query?: string;
        success?: () => void;
        fail?: (err: unknown) => void;
    }) => void;
    getPrivacySetting?: (options: {
        success?: (res: MiniGamePrivacySettingResult) => void;
        fail?: (err: unknown) => void;
    }) => void;
    requirePrivacyAuthorize?: (options: {
        success?: () => void;
        fail?: (err: unknown) => void;
        complete?: () => void;
    }) => void;
    openPrivacyContract?: (options: {
        success?: () => void;
        fail?: (err: unknown) => void;
    }) => void;
    checkScene?: (options: {
        scene: 'sidebar';
        success?: (res: MiniGameSceneCheckResult) => void;
        fail?: (err: unknown) => void;
        complete?: () => void;
    }) => void;
    navigateToScene?: (options: {
        scene: 'sidebar';
        success?: () => void;
        fail?: (err: unknown) => void;
        complete?: () => void;
    }) => void;
    onShow?: (callback: (res: MiniGameShowOptions) => void) => void;
    getLaunchOptionsSync?: () => MiniGameShowOptions;
    getOpenDataContext?: () => MiniGameOpenDataContext;
    createPageManager?: () => MiniGamePageManager;
};
```

```
export class PlatformManager {
    private static _instance: PlatformManager;

    private _initialized: boolean = false;
    private _currentPlatform: MiniGamePlatform = MiniGamePlatform.Auto;
    private _currentRuntime: MiniGameRuntime | null = null;
    private _latestShowOptions: MiniGameShowOptions | null = null;

    static getInstance(): PlatformManager {
        if (!this._instance) {
            this._instance = new PlatformManager();
        }
        return this._instance;
    }

    init(): void {
        if (this._initialized) return;

        this._currentPlatform = this.resolvePlatformBySystem();
        this._currentRuntime = this.getRuntime(this._currentPlatform);
        this.registerShowListener(this._currentPlatform, this._currentRuntime);
        this._initialized = true;
    }

    getPlatform(): MiniGamePlatform {
        if (!this._initialized) this.init();
        return this._currentPlatform;
    }

    async login(platform: MiniGamePlatform = MiniGamePlatform.Auto): Promise<PlatformLoginResult> {
        if (!this._initialized) this.init();

        const resolvedPlatform = this.resolvePlatform(platform);
        const runtime = this.getRuntimeByPlatform(resolvedPlatform);
        if (resolvedPlatform === MiniGamePlatform.Auto || !runtime?.login) {
            return {
                result: PlatformResult.Unsupported,
                platform: resolvedPlatform,
                message: '当前环境不支持小游戏登录',
            };
        }

        try {
            const loginResult = await this.callLogin(runtime);
            const userInfoResult = await this.callGetUserInfo(runtime);
            return {
                result: PlatformResult.Success,
                platform: resolvedPlatform,
                code: loginResult.code,
                userInfo: userInfoResult?.userInfo,
```

```
            message: '登录成功',
        };
    } catch (err) {
        return {
            result: PlatformResult.Failed,
            platform: resolvedPlatform,
            message: '登录失败',
            err,
        };
    }
}

async getPrivacySetting(platform: MiniGamePlatform = MiniGamePlatform.Auto): Promise<PlatformPrivacySettingResult> {
    if (!this._initialized) this.init();

    const resolvedPlatform = this.resolvePlatform(platform);
    const runtime = this.getRuntimeByPlatform(resolvedPlatform);
    if (resolvedPlatform !== MiniGamePlatform.WeChat || !runtime?.getPrivacySetting) {
        const result: PlatformPrivacySettingResult = {
            result: PlatformResult.Unsupported,
            platform: resolvedPlatform,
            needAuthorization: false,
            message: '当前环境不支持隐私授权状态查询',
        };
        this.logPrivacyStep('查询隐私授权状态', result);
        return result;
    }

    return new Promise(resolve => {
        runtime.getPrivacySetting({
            success: (res) => {
                const result: PlatformPrivacySettingResult = {
                    result: PlatformResult.Success,
                    platform: resolvedPlatform,
                    needAuthorization: !!res.needAuthorization,
                    privacyContractName: res.privacyContractName,
                  message: res.needAuthorization ? '需要用户同意隐私协议' : '已同步隐私授权状态',
                };
                this.logPrivacyStep('查询隐私授权状态', result);
                resolve(result);
            },
            fail: (err: unknown) => {
                const result: PlatformPrivacySettingResult = {
                    result: PlatformResult.Failed,
                    platform: resolvedPlatform,
                    message: '隐私授权状态查询失败',
                    err,
                };
                this.logPrivacyStep('查询隐私授权状态', result);
                resolve(result);
```

```
            },
        });
    });
}

async requirePrivacyAuthorize(platform: MiniGamePlatform = MiniGamePlatform.Auto): Promise<PlatformPrivacyAuthorizeResult> {
    if (!this._initialized) this.init();

    const resolvedPlatform = this.resolvePlatform(platform);
    const runtime = this.getRuntimeByPlatform(resolvedPlatform);
    if (resolvedPlatform !== MiniGamePlatform.WeChat || !runtime?.requirePrivacyAuthorize) {
        const result: PlatformPrivacyAuthorizeResult = {
            result: PlatformResult.Unsupported,
            platform: resolvedPlatform,
            message: '当前环境不支持隐私授权弹窗',
        };
        this.logPrivacyStep('拉起隐私授权弹窗', result);
        return result;
    }

    return new Promise(resolve => {
        runtime.requirePrivacyAuthorize({
            success: () => {
                const result: PlatformPrivacyAuthorizeResult = {
                    result: PlatformResult.Success,
                    platform: resolvedPlatform,
                    message: '隐私授权成功',
                };
                this.logPrivacyStep('拉起隐私授权弹窗', result);
                resolve(result);
            },
            fail: (err: unknown) => {
                const result: PlatformPrivacyAuthorizeResult = {
                    result: PlatformResult.Failed,
                    platform: resolvedPlatform,
                    message: '隐私授权失败',
                    err,
                };
                this.logPrivacyStep('拉起隐私授权弹窗', result);
                resolve(result);
            },
        });
    });
}

async openPrivacyContract(platform: MiniGamePlatform = MiniGamePlatform.Auto): Promise<PlatformPrivacyAuthorizeResult> {
    if (!this._initialized) this.init();

    const resolvedPlatform = this.resolvePlatform(platform);
    const runtime = this.getRuntimeByPlatform(resolvedPlatform);
```

```
    if (resolvedPlatform !== MiniGamePlatform.WeChat || !runtime?.openPrivacyContract) {
        const result: PlatformPrivacyAuthorizeResult = {
            result: PlatformResult.Unsupported,
            platform: resolvedPlatform,
            message: '当前环境不支持展示隐私协议',
        };
        this.logPrivacyStep('展示隐私协议', result);
        return result;
    }

    return new Promise(resolve => {
        runtime.openPrivacyContract({
            success: () => {
                const result: PlatformPrivacyAuthorizeResult = {
                    result: PlatformResult.Success,
                    platform: resolvedPlatform,
                    message: '已展示隐私协议',
                };
                this.logPrivacyStep('展示隐私协议', result);
                resolve(result);
            },
            fail: (err: unknown) => {
                const result: PlatformPrivacyAuthorizeResult = {
                    result: PlatformResult.Failed,
                    platform: resolvedPlatform,
                    message: '展示隐私协议失败',
                    err,
                };
                this.logPrivacyStep('展示隐私协议', result);
                resolve(result);
            },
        });
    });
}

async ensurePrivacyAuthorize(platform: MiniGamePlatform = MiniGamePlatform.Auto): Promise<PlatformPrivacyAuthorizeResult> {
    const setting = await this.getPrivacySetting(platform);
    if (setting.result === PlatformResult.Unsupported || setting.needAuthorization === false) {
        const result: PlatformPrivacyAuthorizeResult = {
            result: PlatformResult.Success,
            platform: setting.platform,
            message: setting.message,
        };
        this.logPrivacyStep('隐私授权流程完成', result);
        return result;
    }
    if (setting.result !== PlatformResult.Success) {
        const result: PlatformPrivacyAuthorizeResult = {
            result: setting.result,
            platform: setting.platform,
```

```
            message: setting.message,
            err: setting.err,
        };
        this.logPrivacyStep('隐私授权流程中断', result);
        return result;
    }

    const result = await this.requirePrivacyAuthorize(setting.platform);
    this.logPrivacyStep('隐私授权流程完成', result);
    return result;
}

async checkSidebarScene(platform: MiniGamePlatform = MiniGamePlatform.Auto): Promise<PlatformSidebarResult> {
    if (!this._initialized) this.init();

    const resolvedPlatform = this.resolvePlatform(platform);
    const runtime = this.getRuntimeByPlatform(resolvedPlatform);
    if (resolvedPlatform !== MiniGamePlatform.Douyin || !runtime?.checkScene) {
        return {
            result: PlatformResult.Unsupported,
            platform: resolvedPlatform,
            isExist: false,
            message: '当前环境不支持抖音侧边栏入口检测',
        };
    }

    return new Promise(resolve => {
        runtime.checkScene({
            scene: 'sidebar',
            success: (res) => {
                resolve({
                    result: PlatformResult.Success,
                    platform: resolvedPlatform,
                    isExist: !!res.isExist,
                   message: res.isExist ? '当前宿主支持侧边栏复访' : '当前宿主不支持侧边栏复访',
                });
            },
            fail: (err: unknown) => {
                resolve({
                    result: PlatformResult.Failed,
                    platform: resolvedPlatform,
                    isExist: false,
                    message: '抖音侧边栏入口检测失败',
                    err,
                });
            },
        });
    });
}
```

```
            return true;
        }
    }
    return false;
}

private updateFenceLabels(): void {
    this.m_FenceList.forEach(fence => {
        if (fence.removed) return;
        this.updateFenceLabel(fence);
    });
}

private updateFenceLabel(fence: FenceData): void {
    if (!fence.label || !fence.label.isValid) return;
    const remain = Math.max(0, fence.eliminateCount - this.m_EliminatedCount);
    fence.label.string = `${remain}`;
}

private getAvailableSheepCount(): number {
  return this.m_SheepList.filter(sheep => !sheep.removed && !sheep.moving && !this.isSheepFenced(sheep)).length;
}

private clearFences(): void {
    this.m_FenceList.forEach(fence => {
        if (fence.node && fence.node.isValid) {
            tween(fence.node).stop();
            fence.node.removeFromParent();
            fence.node.destroy();
        }
    });
    this.m_FenceList = [];
}

private resolveSheepType(type?: string): string {
    return type || DEFAULT_SHEEP_TYPE;
}

private resolveSheepDirection(direction: SheepDirection | string): SheepDirection | null {
    if (typeof direction === 'number' && DirectionConfigs[direction as SheepDirection]) {
        return direction as SheepDirection;
    }

    if (typeof direction !== 'string') return null;

    const normalizedDirection = direction.toLowerCase();
    if (normalizedDirection === 'up') return SheepDirection.Up;
    if (normalizedDirection === 'right') return SheepDirection.Right;
    if (normalizedDirection === 'down') return SheepDirection.Down;
    if (normalizedDirection === 'left') return SheepDirection.Left;
```

```
    return null;
}

private createSheep(row: number, col: number, direction: SheepDirection, type: string): boolean {
    if (!this.canPlaceSheep(row, col, direction, type)) return false;
    const spriteFrame = this.getSheepSpriteFrame(type);
    if (!spriteFrame) return false;

    const node = new Node(`Sheep_${row}_${col}`);
    node.layer = this.m_GameRoot.layer;
    this.m_GameRoot.addChild(node);
    node.setPosition(this.getSheepPosition(row, col, direction, type));

    const transform = node.addComponent(UITransform);

    const sprite = node.addComponent(Sprite);
    sprite.spriteFrame = spriteFrame;
    sprite.sizeMode = Sprite.SizeMode.TRIMMED;

    const spriteRect = spriteFrame.rect;
    transform.setContentSize(spriteRect.width, spriteRect.height);
    node.setScale(this.m_SheepScale, this.m_SheepScale, 1);

    const button = node.addComponent(Button);
    this.SetBtnEvent(button, () => this.onSheepClick(sheep));

    const footprint = this.getSheepFootprint(direction, type);

    const sheep: SheepData = {
        node,
        row,
        col,
        rowSpan: footprint.rowSpan,
        colSpan: footprint.colSpan,
        direction,
        type,
        moving: false,
        removed: false,
    };

    this.applySheepDirection(sheep, direction);
    this.setSheepGrid(sheep, sheep);
    this.m_SheepList.push(sheep);
    return true;
}

private getSheepSpriteFrame(type: string): SpriteFrame | null {
    return this.m_SheepSpriteFrameMap.get(type)
        || this.m_SheepSpriteFrameMap.get(DEFAULT_SHEEP_TYPE)
```

```
        || null;
}

private onSheepClick(sheep: SheepData): void {
  if (!sheep || sheep.removed || sheep.moving || this.m_LevelEnded || this.m_IsPaused || this.m_IsTransitioning) return;
    if (this.isSheepFenced(sheep)) return;

    if (this.m_SkillMode === 'removeTwo') {
        this.removeSheepBySkill(sheep);
        return;
    }

    if (this.m_SkillMode === 'flipOne') {
        this.flipSheep(sheep);
        this.m_SkillMode = 'none';
        return;
    }

    this.moveSheep(sheep);
}

private moveSheep(sheep: SheepData): void {
    const moveResult = this.getMoveResult(sheep);
    const fromRow = sheep.row;
    const fromCol = sheep.col;
    const targetPosition = moveResult.blocked
       ? this.getSheepPosition(moveResult.targetRow, moveResult.targetCol, sheep.direction, sheep.type)
        : this.getRunOutPosition(sheep);

    sheep.moving = true;
    this.clearSheepGrid(sheep);
    if (moveResult.blocked) {
        sheep.row = moveResult.targetRow;
        sheep.col = moveResult.targetCol;
        this.setSheepGrid(sheep, sheep);
    } else {
        sheep.removed = true;
    }

    tween(sheep.node)
     .to(Math.max(MIN_MOVE_DURATION, moveResult.distanceCell * MOVE_DURATION_PER_CELL), { position: targetPosition })
        .call(() => {
            if (moveResult.blocked) {
                this.playSheepHitAnimation(sheep, () => {
                    sheep.moving = false;
                });
            } else {
                sheep.moving = false;
                this.destroySheep(sheep);
                this.onSheepEliminated();
```

```
                this.checkLevelEnd();
            }
        })
        .start();
}

private playSheepHitAnimation(sheep: SheepData, onComplete: () => void): void {
    if (!sheep.node || !sheep.node.isValid) {
        onComplete();
        return;
    }

    const originScale = sheep.node.scale.clone();
    const hitScale = new Vec3(originScale.x * 1.12, originScale.y * 0.88, originScale.z);
    tween(sheep.node)
        .to(0.08, { scale: hitScale })
        .to(0.08, { scale: originScale })
        .call(onComplete)
        .start();
}

private getMoveResult(sheep: SheepData): SheepMoveResult {
    const config = DirectionConfigs[sheep.direction];
    let targetRow = sheep.row;
    let targetCol = sheep.col;
    let nextRow = sheep.row + config.rowDelta;
    let nextCol = sheep.col + config.colDelta;

    while (this.isFootprintInside(nextRow, nextCol, sheep.rowSpan, sheep.colSpan)) {
        if (!this.canPlaceFootprint(nextRow, nextCol, sheep.rowSpan, sheep.colSpan, sheep)
            || this.isFootprintBlockedByFence(nextRow, nextCol, sheep.rowSpan, sheep.colSpan)) {
            return {
                blocked: true,
                targetRow,
                targetCol,
               distanceCell: Math.abs(targetRow - sheep.row) + Math.abs(targetCol - sheep.col),
            };
        }

        targetRow = nextRow;
        targetCol = nextCol;
        nextRow += config.rowDelta;
        nextCol += config.colDelta;
    }

    return {
        blocked: false,
        targetRow,
        targetCol,
        distanceCell: this.getRunOutDistanceCell(sheep),
```

```
    };
}

private onPauseBtnClick(): void {
    if (this.m_LevelEnded || this.m_IsPaused || this.m_IsTransitioning) return;

    this.m_IsPaused = true;
    UIManager.GetInstance().OpenPanel(CommonUIID.PausePanel, {
        onContinue: () => this.continueCurrentLevel(),
        onRestart: () => this.restartCurrentLevel(),
        onGoBack: () => this.goBackMainPanel(),
    });
}

private continueCurrentLevel(): void {
    this.m_IsPaused = false;
}

private restartCurrentLevel(): void {
    this.m_IsPaused = false;
    this.loadLevel(this.m_CurrentLevel);
}

private goBackMainPanel(): void {
    this.m_IsPaused = false;
    UIManager.GetInstance().OpenPanelWithCallback(CommonUIID.MainPanel, () => {
        UIManager.GetInstance().ClosePanel(CommonUIID.GamePanel);
    });
}

private onSkillOneBtnClick(): void {
   if (this.m_LevelEnded || this.m_IsPaused || this.m_IsTransitioning || this.m_SheepList.length <= 0) return;
    this.openSkillAdPanel(1);
}

private onSkillTwoBtnClick(): void {
   if (this.m_LevelEnded || this.m_IsPaused || this.m_IsTransitioning || this.m_SheepList.length <= 0) return;
    if (this.getAvailableSheepCount() <= 0) return;
    this.openSkillAdPanel(2);
}

private onSkillThreeBtnClick(): void {
   if (this.m_LevelEnded || this.m_IsPaused || this.m_IsTransitioning || this.m_SheepList.length <= 0) return;
    if (this.getAvailableSheepCount() <= 0) return;
    this.openSkillAdPanel(3);
}

private openSkillAdPanel(skillIndex: AdSkillIndex): void {
    UIManager.GetInstance().OpenPanel(CommonUIID.AdPanel, {
        skillIndex,
```

```
        onUse: () => this.applySkill(skillIndex),
        shareQuery: `level=${this.m_CurrentLevel}`,
    });
}

private applySkill(skillIndex: AdSkillIndex): void {
    if (!this.isValid || this.m_LevelEnded || this.m_IsPaused || this.m_IsTransitioning) return;

    if (skillIndex === 1) {
        this.m_SkillMode = 'removeTwo';
        this.m_SkillRemoveRemain = Math.min(2, this.getAvailableSheepCount());
        return;
    }

    if (skillIndex === 2) {
        this.getRandomSheepList(Math.min(5, this.getAvailableSheepCount())).forEach(sheep => {
            this.flipSheep(sheep);
        });
        return;
    }

    this.m_SkillMode = 'flipOne';
    this.m_SkillRemoveRemain = 0;
}

private removeSheepBySkill(sheep: SheepData): void {
    if (this.m_SkillRemoveRemain <= 0) {
        this.m_SkillMode = 'none';
        return;
    }

    sheep.removed = true;
    this.clearSheepGrid(sheep);
    this.m_SkillRemoveRemain--;

    tween(sheep.node)
        .to(0.18, { scale: new Vec3(0, 0, 1) })
        .call(() => {
            this.destroySheep(sheep);
            this.onSheepEliminated();
            this.checkLevelEnd();
        })
        .start();

    if (this.m_SkillRemoveRemain <= 0) {
        this.m_SkillMode = 'none';
    }
}

private onSheepEliminated(): void {
```

```
    this.m_EliminatedCount++;
    this.updateFenceLabels();
    this.checkFenceRemoval();
}

private checkFenceRemoval(): void {
    this.m_FenceList.forEach(fence => {
        if (fence.removed) return;
        if (this.m_EliminatedCount >= fence.eliminateCount) {
            this.removeFence(fence);
        }
    });
}

private removeFence(fence: FenceData): void {
    fence.removed = true;
    if (!fence.node || !fence.node.isValid) return;

    tween(fence.node)
        .to(0.25, { scale: new Vec3(0, 0, 1) })
        .call(() => {
            if (fence.node && fence.node.isValid) {
                fence.node.removeFromParent();
                fence.node.destroy();
            }
        })
        .start();
}

private flipSheep(sheep: SheepData): void {
    if (!sheep || sheep.removed || sheep.moving) return;

    const direction = (sheep.direction + 2) % 4 as SheepDirection;
    this.applySheepDirection(sheep, direction);
}

private applySheepDirection(sheep: SheepData, direction: SheepDirection): void {
    sheep.direction = direction;
    sheep.node.angle = DirectionConfigs[direction].angle;
}

private getRandomSheepList(count: number): SheepData[] {
  const availableList = this.m_SheepList.filter(sheep => !sheep.removed && !sheep.moving && !this.isSheepFenced(sheep));
    for (let i = availableList.length - 1; i > 0; i--) {
        const randomIndex = Math.floor(Math.random() * (i + 1));
        const temp = availableList[i];
        availableList[i] = availableList[randomIndex];
        availableList[randomIndex] = temp;
    }
```

```
    return availableList.slice(0, count);
}

private getSheepPosition(row: number, col: number, direction: SheepDirection, type: string): Vec3 {
    const footprint = this.getSheepFootprint(direction, type);
    const x = this.m_BoardLeft + this.m_CellWidth * (col + footprint.colSpan * 0.5);
    const y = this.m_BoardTop - this.m_CellHeight * (row + footprint.rowSpan * 0.5);
    return new Vec3(x, y, 0);
}

private getRunOutPosition(sheep: SheepData): Vec3 {
    const config = DirectionConfigs[sheep.direction];
    const position = sheep.node.position.clone();
    const distance = this.getRunOutDistanceCell(sheep);
    return new Vec3(
        position.x + config.moveX * this.m_CellWidth * distance,
        position.y + config.moveY * this.m_CellHeight * distance,
        position.z,
    );
}

private getRunOutDistanceCell(sheep: SheepData): number {
   if (sheep.direction === SheepDirection.Up) return sheep.row + sheep.rowSpan + RUN_OUT_EXTRA_CELL;
   if (sheep.direction === SheepDirection.Down) return this.m_RowCount - sheep.row + RUN_OUT_EXTRA_CELL;
   if (sheep.direction === SheepDirection.Left) return sheep.col + sheep.colSpan + RUN_OUT_EXTRA_CELL;
    return this.m_ColCount - sheep.col + RUN_OUT_EXTRA_CELL;
}

private isInsideGrid(row: number, col: number): boolean {
    return row >= 0 && row < this.m_RowCount && col >= 0 && col < this.m_ColCount;
}

private getSheepFootprint(direction: SheepDirection, type: string): SheepFootprint {
    const typeConfig = this.m_SheepTypeConfigMap.get(type) || this.getDefaultSheepTypeConfig();
    if (this.isHorizontalDirection(direction)) {
        return typeConfig.horizontal || { rowSpan: 1, colSpan: 2 };
    }

    return typeConfig.vertical || { rowSpan: 2, colSpan: 1 };
}

private isHorizontalDirection(direction: SheepDirection): boolean {
    return direction === SheepDirection.Left || direction === SheepDirection.Right;
}

private canPlaceSheep(row: number, col: number, direction: SheepDirection, type: string): boolean {
    const footprint = this.getSheepFootprint(direction, type);
    return this.canPlaceFootprint(row, col, footprint.rowSpan, footprint.colSpan, null);
}
```

```
private canPlaceFootprint(row: number, col: number, rowSpan: number, colSpan: number, ignoreSheep: SheepData | null): boolean {
    if (!this.isFootprintInside(row, col, rowSpan, colSpan)) return false;

    for (let rowOffset = 0; rowOffset < rowSpan; rowOffset++) {
        for (let colOffset = 0; colOffset < colSpan; colOffset++) {
            const target = this.m_Grid[row + rowOffset][col + colOffset];
            if (target && target !== ignoreSheep && !target.removed) return false;
        }
    }

    return true;
}

private isFootprintInside(row: number, col: number, rowSpan: number, colSpan: number): boolean {
   return row >= 0 && col >= 0 && row + rowSpan <= this.m_RowCount && col + colSpan <= this.m_ColCount;
}

private setSheepGrid(sheep: SheepData, value: SheepData | null): void {
    for (let rowOffset = 0; rowOffset < sheep.rowSpan; rowOffset++) {
        for (let colOffset = 0; colOffset < sheep.colSpan; colOffset++) {
            const row = sheep.row + rowOffset;
            const col = sheep.col + colOffset;
            if (this.isInsideGrid(row, col)) {
                this.m_Grid[row][col] = value;
            }
        }
    }
}

private clearSheepGrid(sheep: SheepData): void {
    this.setSheepGrid(sheep, null);
}

private clampInt(value: number, min: number, max: number): number {
    return Math.floor(this.clampNumber(value, min, max));
}

private clampNumber(value: number, min: number, max: number): number {
    if (Number.isNaN(value)) return min;

    return Math.min(Math.max(value, min), max);
}

private destroySheep(sheep: SheepData): void {
    const index = this.m_SheepList.indexOf(sheep);
    if (index >= 0) {
        this.m_SheepList.splice(index, 1);
    }

    if (sheep.node && sheep.node.isValid) {
```

```
            tween(sheep.node).stop();
            sheep.node.removeFromParent();
            sheep.node.destroy();
        }
    }

    private checkLevelEnd(): void {
        if (this.m_LevelEnded || this.m_SheepList.length > 0) return;

        this.m_LevelEnded = true;
        this.m_SkillMode = 'none';
        this.m_SkillRemoveRemain = 0;
        console.log('GamePanel: 关卡结束');
        this.submitRankScore();
        this.startNextLevel();
    }

    private startNextLevel(): void {
        const nextLevel = this.m_CurrentLevel + 1;
        this.loadLevel(nextLevel);
    }

    private async submitRankScore(): Promise<void> {
       const result = await PlatformManager.getInstance().submitRankScore('level', this.m_CurrentLevel);
        if (result.result !== PlatformResult.Success) {
            console.warn('GamePanel: 排行榜提交失败或不支持', result);
        }
    }

    private clearSheep(): void {
        this.m_SheepList.forEach(sheep => {
            if (sheep.node && sheep.node.isValid) {
                tween(sheep.node).stop();
                sheep.node.removeFromParent();
                sheep.node.destroy();
            }
        });
        this.m_SheepList = [];
        this.m_Grid = [];
    }
}
import { _decorator, Prefab, ProgressBar } from 'cc';
import { ResManager } from '../../../engine/ResManager';
import { UIBase } from '../../../engine/ui/UIBase';
import { UIManager } from '../../../engine/ui/UIManager';
import { FrameworkConst } from '../../../framework/FrameworkConst';
import { CommonBundleName, CommonUIID } from '../CommonUIConfig';
const { ccclass, property } = _decorator;

@ccclass('LoadingPanel')
```

```
export class LoadingPanel extends UIBase {
    @property(ProgressBar)
    m_Progress: ProgressBar = null;

    private m_NextPanelID: number = CommonUIID.LoginPanel;
    private m_NextPanelArgs: any[] = [];
    private m_LoadingSerial: number = 0;
    private m_IsLoading: boolean = false;
    private m_MinDuration: number = FrameworkConst.LOADING_DURATION;
    private m_ElapsedTime: number = 0;
    private m_TargetProgress: number = 0;
    private m_DisplayProgress: number = 0;
    private m_LoadComplete: boolean = false;

    OnInit(): void {}

    OnOpen(nextPanelID: number = CommonUIID.LoginPanel, duration: number = FrameworkConst.LOADING_DURATION, ...nextPanelArgs: any[]): void {
        this.m_NextPanelID = nextPanelID;
        this.m_NextPanelArgs = nextPanelArgs;
        this.m_MinDuration = duration > 0 ? duration : FrameworkConst.LOADING_DURATION;
        this.m_ElapsedTime = 0;
        this.m_TargetProgress = 0;
        this.m_DisplayProgress = 0;
        this.m_LoadComplete = false;
        if (this.m_Progress) {
            this.m_Progress.progress = 0;
        }
        this.startPreloadBundles();
    }

    OnClose(): void {
        super.OnClose();
        this.m_LoadingSerial++;
        this.m_IsLoading = false;
        this.m_LoadComplete = false;
        this.m_NextPanelArgs = [];
    }

    protected update(dt: number): void {
        if (!this.m_IsLoading && !this.m_LoadComplete) return;

        this.m_ElapsedTime += dt;
        const timeProgress = Math.min(this.m_ElapsedTime / this.m_MinDuration, 1);
        const cappedTargetProgress = this.m_LoadComplete
            ? timeProgress
            : Math.min(this.m_TargetProgress, timeProgress);
        this.m_DisplayProgress = Math.max(this.m_DisplayProgress, cappedTargetProgress);

        if (this.m_Progress) {
            this.m_Progress.progress = this.m_DisplayProgress;
```

```
    }

    if (this.m_LoadComplete && this.m_DisplayProgress >= 1) {
        this.openNextPanel();
    }
}

private startPreloadBundles(): void {
    if (this.m_IsLoading) return;

    this.m_IsLoading = true;
    const serial = ++this.m_LoadingSerial;
    const bundles = [CommonBundleName.Game, CommonBundleName.Rank];
    const entries = [
        { bundleName: CommonBundleName.Game, path: 'ui/GamePanel', type: Prefab },
        { bundleName: CommonBundleName.Game, path: 'ui/PausePanel', type: Prefab },
        { bundleName: CommonBundleName.Game, path: 'ui/TransitionPanel', type: Prefab },
        { bundleName: CommonBundleName.Game, path: 'ui/AdPanel', type: Prefab },
        { bundleName: CommonBundleName.Rank, path: 'ui/RankPanel', type: Prefab },
    ];

    ResManager.getInstance().preloadBundles(
        bundles,
        entries,
        (_finished, _total, progress) => {
            if (serial !== this.m_LoadingSerial || !this.isValid) return;
            this.m_TargetProgress = Math.max(this.m_TargetProgress, progress);
        },
        err => {
            if (serial !== this.m_LoadingSerial || !this.isValid) return;
            this.m_IsLoading = false;
            if (err) {
                console.warn('LoadingPanel: 加载分包失败', err);
                return;
            }

            this.m_TargetProgress = 1;
            this.m_LoadComplete = true;
        },
    );
}

private openNextPanel(): void {
    this.m_LoadComplete = false;
    const nextPanelID = this.m_NextPanelID;
    const nextPanelArgs = this.m_NextPanelArgs;
    if (this.m_Progress) {
        this.m_Progress.progress = 1;
    }
    UIManager.GetInstance().ClosePanel(this.m_UIID || CommonUIID.LoadingPanel);
```

```
        UIManager.GetInstance().OpenPanel(nextPanelID, ...nextPanelArgs);
    }
}
import { _decorator, Button, Prefab, ProgressBar } from 'cc';
import { PlatformManager, PlatformResult } from '../../../engine/PlatformManager';
import { ResManager } from '../../../engine/ResManager';
import { UIBase } from '../../../engine/ui/UIBase';
import { UIManager } from '../../../engine/ui/UIManager';
import { FrameworkConst } from '../../../framework/FrameworkConst';
import { CommonBundleName, CommonUIID } from '../CommonUIConfig';
const { ccclass, property } = _decorator;

@ccclass('LoginPanel')
export class LoginPanel extends UIBase {
    @property(ProgressBar)
    m_Progress: ProgressBar = null;

    private m_EnterButton: Button = null;
    private m_PrivacyButton: Button = null;
    private m_IsLoggingIn: boolean = false;
    private m_LoginSerial: number = 0;
    private m_LoadingSerial: number = 0;
    private m_IsLoadingBundles: boolean = false;
    private m_BundlesLoaded: boolean = false;
    private m_MinDuration: number = FrameworkConst.LOADING_DURATION;
    private m_ElapsedTime: number = 0;
    private m_TargetProgress: number = 0;
    private m_DisplayProgress: number = 0;
    private m_LoadComplete: boolean = false;

    OnInit(): void {}

    OnOpen(): void {
        this.ensureEnterButton();
        this.ensurePrivacyButton();
        this.setEnterButtonVisible(false);
        this.setPrivacyButtonVisible(false);
        this.setProgressVisible(true);
        this.startPreloadBundles();
    }

    OnClose(): void {
        super.OnClose();
        this.m_LoginSerial++;
        this.m_LoadingSerial++;
        this.m_IsLoggingIn = false;
        this.m_IsLoadingBundles = false;
        this.m_LoadComplete = false;
    }
```

```
protected update(dt: number): void {
    if (!this.m_IsLoadingBundles && !this.m_LoadComplete) return;

    this.m_ElapsedTime += dt;
    const timeProgress = Math.min(this.m_ElapsedTime / this.m_MinDuration, 1);
    const cappedTargetProgress = this.m_LoadComplete
        ? timeProgress
        : Math.min(this.m_TargetProgress, timeProgress);
    this.m_DisplayProgress = Math.max(this.m_DisplayProgress, cappedTargetProgress);
    this.setProgress(this.m_DisplayProgress);

    if (this.m_LoadComplete && this.m_DisplayProgress >= 1) {
        this.finishPreloadAndLogin();
    }
}

private startPreloadBundles(): void {
    if (this.m_BundlesLoaded) {
        this.finishPreloadAndLogin();
        return;
    }
    if (this.m_IsLoadingBundles) return;

    this.m_IsLoadingBundles = true;
    this.m_LoadComplete = false;
    this.m_ElapsedTime = 0;
    this.m_TargetProgress = 0;
    this.m_DisplayProgress = 0;
    this.setProgressVisible(true);
    this.setProgress(0);

    const serial = ++this.m_LoadingSerial;
    const bundles = [CommonBundleName.Game, CommonBundleName.Rank];
    const entries = [
        { bundleName: CommonBundleName.Game, path: 'ui/GamePanel', type: Prefab },
        { bundleName: CommonBundleName.Game, path: 'ui/PausePanel', type: Prefab },
        { bundleName: CommonBundleName.Game, path: 'ui/TransitionPanel', type: Prefab },
        { bundleName: CommonBundleName.Game, path: 'ui/AdPanel', type: Prefab },
        { bundleName: CommonBundleName.Rank, path: 'ui/RankPanel', type: Prefab },
    ];

    ResManager.getInstance().preloadBundles(
        bundles,
        entries,
        (_finished, _total, progress) => {
            if (serial !== this.m_LoadingSerial || !this.isValid) return;
            this.m_TargetProgress = Math.max(this.m_TargetProgress, progress);
        },
        err => {
            if (serial !== this.m_LoadingSerial || !this.isValid) return;
```

```
            this.m_IsLoadingBundles = false;
            if (err) {
                console.warn('LoginPanel: 加载分包失败', err);
                this.setEnterButtonVisible(true);
                return;
            }

            this.m_TargetProgress = 1;
            this.m_LoadComplete = true;
        },
    );
}

private finishPreloadAndLogin(): void {
    this.m_LoadComplete = false;
    this.m_BundlesLoaded = true;
    this.setProgress(1);
    this.setProgressVisible(false);
    this.login();
}

private async login(): Promise<void> {
    if (this.m_IsLoggingIn) return;

    this.m_IsLoggingIn = true;
    const serial = ++this.m_LoginSerial;
    this.setEnterButtonVisible(false);

    const privacyResult = await PlatformManager.getInstance().ensurePrivacyAuthorize();
    if (serial !== this.m_LoginSerial || !this.isValid) return;
    if (privacyResult.result !== PlatformResult.Success) {
        this.m_IsLoggingIn = false;
        console.warn('LoginPanel: 隐私授权失败', privacyResult);
        this.setEnterButtonVisible(true);
        return;
    }

    const result = await PlatformManager.getInstance().login();
    if (serial !== this.m_LoginSerial || !this.isValid) return;

    this.m_IsLoggingIn = false;
    if (result.result === PlatformResult.Success || result.result === PlatformResult.Unsupported) {
        console.log('LoginPanel: 进入主界面', result);
        UIManager.GetInstance().OpenPanelWithCallback(CommonUIID.MainPanel, () => {
            UIManager.GetInstance().ClosePanel(CommonUIID.LoginPanel);
        });
        return;
    }

    console.warn('LoginPanel: 登录失败', result);
```

```
    this.setEnterButtonVisible(true);
}

private ensureEnterButton(): void {
    if (this.m_EnterButton && this.m_EnterButton.isValid) return;

    this.m_EnterButton = this.CreateUIButton(
        this.node,
        'EnterGame',
        '进入游戏',
        'buttons/Button01_145_Orange',
        () => {
            if (this.m_BundlesLoaded) {
                this.login();
                return;
            }
            this.startPreloadBundles();
        },
    );
    if (this.m_EnterButton) {
        this.m_EnterButton.node.setPosition(0, -300, 0);
    }
}

private ensurePrivacyButton(): void {
    if (this.m_PrivacyButton && this.m_PrivacyButton.isValid) return;

    this.m_PrivacyButton = this.CreateUIButton(
        this.node,
        'PrivacyContract',
        '隐私协议',
        'buttons/Button01_145_Orange',
        () => this.openPrivacyContract(),
    );
    if (this.m_PrivacyButton) {
        this.m_PrivacyButton.node.setPosition(0, -420, 0);
        this.setPrivacyButtonVisible(false);
    }
}

private async openPrivacyContract(): Promise<void> {
    const result = await PlatformManager.getInstance().openPrivacyContract();
    if (result.result !== PlatformResult.Success) {
        console.warn('LoginPanel: 展示隐私协议失败', result);
    }
}

private setEnterButtonVisible(visible: boolean): void {
    if (this.m_EnterButton && this.m_EnterButton.isValid) {
        this.m_EnterButton.node.active = visible;
```

```
        }
    }

    private setPrivacyButtonVisible(visible: boolean): void {
        if (this.m_PrivacyButton && this.m_PrivacyButton.isValid) {
            this.m_PrivacyButton.node.active = visible;
        }
    }

    private setProgressVisible(visible: boolean): void {
        if (this.m_Progress?.node) {
            this.m_Progress.node.active = visible;
        }
    }

    private setProgress(progress: number): void {
        const clampedProgress = Math.max(0, Math.min(progress, 1));
        if (this.m_Progress) {
            this.m_Progress.progress = clampedProgress;
        }
    }
}
import { _decorator, Node } from 'cc';
import { MiniGamePlatform, PlatformManager, PlatformResult } from '../../../engine/PlatformManager';
import { UIBase } from '../../../engine/ui/UIBase';
import { UIManager } from '../../../engine/ui/UIManager';
import { CommonGameProgress } from '../CommonGameProgress';
import { CommonUIID } from '../CommonUIConfig';
const { ccclass, property } = _decorator;

@ccclass('MainPage')
export class MainPage extends UIBase {
    @property(Node)
    m_TopRoot: Node = null;
    @property(Node)
    m_LeftRoot: Node = null;
    @property(Node)
    m_RightRoot: Node = null;
    @property(Node)
    m_MiddleRoot: Node = null;

    OnInit(): void {}

    OnOpen(): void {
        this.initUI();
    }

    OnClose(): void {
        super.OnClose();
        this.clearRoot(this.m_LeftRoot);
```

```
    this.clearRoot(this.m_RightRoot);
    this.clearRoot(this.m_MiddleRoot);
}

private initUI(): void {
    this.clearRoot(this.m_LeftRoot);
    this.clearRoot(this.m_RightRoot);
    this.clearRoot(this.m_MiddleRoot);

    this.InitUIButtons();
}

private InitUIButtons(): void {
    const buttons = this.CreateUIButtonsByTable(this.m_MiddleRoot, [
        {
            buttonName: "StartGame",
            buttonText: "",
            buttonIcon: "buttons/Button01_145_Orange",
            onClick: () => {
                const currentLevel = CommonGameProgress.getCurrentLevel(1);
                UIManager.GetInstance().OpenPanelWithCallback(CommonUIID.GamePanel, () => {
                    UIManager.GetInstance().ClosePanel(CommonUIID.MainPanel);
                }, currentLevel);
            },
        },
    ]);

    buttons.forEach((button, index) => {
        button.node.setPosition(0, 120 - index * 90, 0);
    });

    const rightButtonConfigs = [
        {
            buttonName: "Rank",
            buttonText: "",
            buttonIcon: "texture/Icon_ImageIcon_Ranking",
            onClick: () => {
                UIManager.GetInstance().OpenPanel(CommonUIID.RankPanel);
            },
        },
    ];


    const leftButtonConfigs = [{
        buttonName: "GameClub",
        buttonText: "",
        buttonIcon: "texture/Icon_ImageIcon_Game",
        onClick: () => {
            this.openWeChatGameClub();
        },
```

```
    }];

    if (PlatformManager.getInstance().getPlatform() === MiniGamePlatform.WeChat) {
        const leftButtons = this.CreateUIButtonsByTable(this.m_LeftRoot, leftButtonConfigs);

        leftButtons.forEach((button, index) => {
            button.node.setPosition(0, 80 - index * 90, 0);
        });
    }

    // if (PlatformManager.getInstance().getPlatform() === MiniGamePlatform.Douyin) {
    //     rightButtonConfigs.push({
    //         buttonName: "Sidebar",
    //         buttonText: "侧边栏",
    //         buttonIcon: "buttons/Icon_ImageIcon_Ranking",
    //         onClick: () => {
    //             this.openDouyinSidebar();
    //         },
    //     });
    // }

    const rightButtons = this.CreateUIButtonsByTable(this.m_RightRoot, rightButtonConfigs);

    rightButtons.forEach((button, index) => {
        button.node.setPosition(0, 80 - index * 90, 0);
    });
}

/** 微信游戏圈 OPENLINK 值（由微信开放平台/渠道下发） */
private static readonly GAME_CLUB_OPEN_LINK = '-SSEykJvFV3pORt5kTNpS2-MczMSa1aUsUcE1ADW8e312BGtUIHxCLNVcvBXDjVOy8e81_K9sVzCb-FNDAK_5OEBVFxe2MmRSnYv27_-Euv2ByeOq9FqwbW_ApJi_MoJ5CbomTGuKDMR1v7whMeOFUGFaganV4W3p14yBJs-ovJrKGZDQa7BwIEcmhUcHA9QzLv4ddvJkmINyixQWsQUad1hi9iUapw42CxLEJ3-Hp76oqp82oGmhd1Us_bnIOrMeYc-c4NTv4KCUkOI8TOHOWUrV1be-QZB8Hq-edxoceJEJADWXJD6g7B8PV7A9vmuQQDsVAXNEpiykZ_EE5cLZg';

private async openWeChatGameClub(): Promise<void> {
    const result = await PlatformManager.getInstance().openGameClubPage(MainPage.GAME_CLUB_OPEN_LINK);
    if (result.result !== PlatformResult.Success) {
        console.warn('MainPage: 拉起微信游戏圈失败', result);
    }
}

private async openDouyinSidebar(): Promise<void> {
    const result = await PlatformManager.getInstance().navigateToSidebar();
    if (result.result !== PlatformResult.Success) {
        console.warn('MainPage: 跳转抖音侧边栏失败', result);
    }
}

private clearRoot(root: Node): void {
    if (!root) return;

    root.children.forEach(node => {
        node.removeFromParent();
```

```
            node.destroy();
        });
    }
}
import { _decorator, Button, CCString, instantiate, Node, Prefab, resources, Tween, tween, Vec3, view } from 'cc';
import { UIBase } from '../../../engine/ui/UIBase';
import { FrameworkConst } from '../../../framework/FrameworkConst';
const { ccclass, property } = _decorator;

@ccclass('MainPanel')
export class MainPanel extends UIBase {
    @property([Button])
    m_FuncBtns: Button[] = [];
    @property([CCString])
    m_PagePrefabPaths: string[] = ["ui/MainPage"];
    @property
    m_DefaultPageIndex: number = 0;
    @property(Node)
    m_PageOne: Node = null;
    @property(Node)
    m_PageTwo: Node = null;
    @property(Node)
    m_BottomRoot: Node = null;

    private m_CurPage: Node = null;
    private m_OtherPage: Node = null;
    private m_LastIndex: number = 0;
    private m_IsScrollingPage: boolean = false;
    private m_ScreenWidth: number = 0;
    private m_CurTween: Tween<Node> = null;
    private m_OtherTween: Tween<Node> = null;

    OnInit(): void {}

    OnOpen(): void {
        this.WaitOpenReady();
        this.m_LastIndex = this.clampPageIndex(this.m_DefaultPageIndex);
        this.initUI();
    }

    OnClose(): void {
        super.OnClose();
        this.stopTweens();
        this.clearPage(this.m_PageOne);
        this.clearPage(this.m_PageTwo);
        this.m_IsScrollingPage = false;
    }

    protected onDestroy(): void {
        this.stopTweens();
```

```
}

private initUI(): void {
    const screenSize = view.getVisibleSize();
    this.m_ScreenWidth = screenSize.width;
    this.m_PageOne?.setPosition(0, 0);
    this.m_PageTwo?.setPosition(this.m_ScreenWidth, 0);
    this.m_CurPage = this.m_PageOne;
    this.m_OtherPage = this.m_PageTwo;
    this.m_IsScrollingPage = false;

    this.clearPage(this.m_PageOne);
    this.clearPage(this.m_PageTwo);

    this.loadInitialPage(() => this.NotifyOpenReady());
    this.refreshBottomRoot();
    this.bindPageButtons();
}

private refreshBottomRoot(): void {
    if (this.m_BottomRoot) {
        this.m_BottomRoot.active = this.m_FuncBtns.length > 0;
    }
}

private loadInitialPage(onLoaded?: () => void): void {
    this.loadPage(this.m_LastIndex, this.m_PageOne, onLoaded);
}

private bindPageButtons(): void {
    this.m_FuncBtns.forEach((button, index) => {
        button.interactable = index < this.m_PagePrefabPaths.length;
        this.SetBtnEvent(button, () => {
          if (!button.interactable || index === this.m_LastIndex || this.m_IsScrollingPage) return;
            this.scrollToPage(index);
        });
    });
}

private scrollToPage(index: number): void {
    this.m_IsScrollingPage = true;
    const size = view.getVisibleSize();
    this.m_ScreenWidth = size.width;

    let moveDis = 0;
    if (this.m_LastIndex < index) {
        this.m_OtherPage.setPosition(this.m_ScreenWidth, 0);
        moveDis = -this.m_ScreenWidth;
    } else {
        this.m_OtherPage.setPosition(-this.m_ScreenWidth, 0);
```

```
        moveDis = this.m_ScreenWidth;
    }

    this.m_LastIndex = index;
    this.loadPage(index, this.m_OtherPage, () => {
        this.m_OtherTween = tween(this.m_OtherPage)
            .by(FrameworkConst.PAGE_SCROLL_DURATION, { position: new Vec3(moveDis, 0, 0) })
            .start();
        this.m_CurTween = tween(this.m_CurPage)
            .by(FrameworkConst.PAGE_SCROLL_DURATION, { position: new Vec3(moveDis, 0, 0) })
            .call(() => {
                const temp = this.m_CurPage;
                this.m_CurPage = this.m_OtherPage;
                this.m_OtherPage = temp;
                this.m_IsScrollingPage = false;
            })
            .start();
    });
}

private loadPage(index: number, root: Node, onLoaded?: () => void): void {
    this.loadPageByPath(this.m_PagePrefabPaths[index], root, onLoaded);
}

private loadPageByPath(prefabPath: string, root: Node, onLoaded?: () => void): void {
    if (!root || !prefabPath) {
        onLoaded?.();
        return;
    }

    resources.load(prefabPath, Prefab, (err, prefab) => {
        if (err || !prefab || !root.isValid) {
            this.m_IsScrollingPage = false;
            onLoaded?.();
            return;
        }

        const pageNode = instantiate(prefab);
        this.addPage(root, pageNode);
        onLoaded?.();
    });
}

private addPage(root: Node, uiNode: Node): void {
    if (!root || !uiNode) return;

    this.clearPage(root);

    const pageScript = uiNode.getComponent(UIBase);
    if (pageScript) {
```

```
            pageScript.OnInit();
            pageScript.OnOpen();
        }
        root.addChild(uiNode);
    }

    private clearPage(root: Node): void {
        if (!root) return;

        root.children.forEach(node => {
            const pageScript = node.getComponent(UIBase);
            if (pageScript) pageScript.OnClose();
            node.removeFromParent();
            node.destroy();
        });
    }

    private clampPageIndex(index: number): number {
        if (this.m_PagePrefabPaths.length === 0) return 0;
        return Math.max(0, Math.min(index, this.m_PagePrefabPaths.length - 1));
    }

    private stopTweens(): void {
        if (this.m_CurTween) this.m_CurTween.stop();
        if (this.m_OtherTween) this.m_OtherTween.stop();
        this.m_CurTween = null;
        this.m_OtherTween = null;
    }
}
import { _decorator, Button, Node } from 'cc';
import { UIBase } from '../../../engine/ui/UIBase';
import { UIManager } from '../../../engine/ui/UIManager';
import { CommonUIID } from '../CommonUIConfig';
const { ccclass } = _decorator;

export interface PausePanelOptions {
    onContinue?: () => void;
    onRestart?: () => void;
    onGoBack?: () => void;
}

@ccclass('PausePanel')
export class PausePanel extends UIBase {
    private m_OnContinue: (() => void) | null = null;
    private m_OnRestart: (() => void) | null = null;
    private m_OnGoBack: (() => void) | null = null;

    OnOpen(options: PausePanelOptions = {}): void {
        this.m_OnContinue = options.onContinue || null;
        this.m_OnRestart = options.onRestart || null;
```

```
    this.m_OnGoBack = options.onGoBack || null;
    this.bindButtons();
}

OnClose(): void {
    super.OnClose();
    this.m_OnContinue = null;
    this.m_OnRestart = null;
    this.m_OnGoBack = null;
}

private bindButtons(): void {
    const background = this.findChildByName(this.node, 'Background');
    if (!background) return;

    this.bindButton(background, 'CloseBtn', () => this.continueGame());
    this.bindButton(background, 'ContinueBtn', () => this.continueGame());
    this.bindButton(background, 'RestartBtn', () => this.restartGame());
    this.bindButton(background, 'GoBackBtn', () => this.goBackMainPanel());
}

private continueGame(): void {
    const onContinue = this.m_OnContinue;
    this.closePausePanel();
    onContinue?.();
}

private restartGame(): void {
    const onRestart = this.m_OnRestart;
    this.closePausePanel();
    onRestart?.();
}

private goBackMainPanel(): void {
    const onGoBack = this.m_OnGoBack;
    this.closePausePanel();
    onGoBack?.();
}

private bindButton(root: Node, name: string, callback: () => void): void {
    const node = this.findChildByName(root, name);
    const button = node?.getComponent(Button) || node?.addComponent(Button);
    if (button) {
        this.SetBtnEvent(button, callback);
    }
}

private closePausePanel(): void {
    if (this.m_UIID) {
        UIManager.GetInstance().ClosePanel(CommonUIID.PausePanel);
```

```
            return;
        }

        this.OnClose();
        this.node.removeFromParent();
        this.node.destroy();
    }

    private findChildByName(root: Node, name: string): Node | null {
        if (!root) return null;
        if (root.name === name) return root;

        for (const child of root.children) {
            const matched = this.findChildByName(child, name);
            if (matched) return matched;
        }

        return null;
    }
}
import { _decorator, Button, Color, Label, Node, Sprite, SpriteFrame, Texture2D, UITransform } from 'cc';
import { PlatformManager, PlatformResult } from '../../../engine/PlatformManager';
import { UIBase } from '../../../engine/ui/UIBase';

const { ccclass } = _decorator;

const RANK_KEY = 'level';
const RANK_VIEW_WIDTH = 500;
const RANK_VIEW_HEIGHT = 760;

@ccclass('RankPanel')
export class RankPanel extends UIBase {
    private m_StatusLabel: Label = null;
    private m_RankSprite: Sprite = null;
    private m_RankTexture: Texture2D = null;
    private m_RefreshTimer: ReturnType<typeof setInterval> = null;
    private m_ShowBackgroundTimer: ReturnType<typeof setTimeout> = null;
    private m_BackgroundNode: Node = null;

    OnOpen(): void {
        this.bindPrefabUI();
        this.showFriendRank();
    }

    OnClose(): void {
        this.stopRefreshRankCanvas();
        this.clearShowBackgroundTimer();
        PlatformManager.getInstance().postOpenDataMessage({ type: 'hideFriendRank' });
        super.OnClose();
    }
```

```
private bindPrefabUI(): void {
    const transform = this.node.getComponent(UITransform) || this.node.addComponent(UITransform);
    transform.setContentSize(750, 1334);

    const background = this.getBackgroundNode();
    this.m_BackgroundNode = background;
    this.setBackgroundVisible(false);
    const closeNode = this.findChildByName(background, 'CloseBtn');
    const closeButton = closeNode?.getComponent(Button) || closeNode?.addComponent(Button);
    if (closeButton) {
        this.SetBtnEvent(closeButton, () => this.closeRankPanel());
    }

    this.bindRankView(background);
    this.hideUnusedPrefabRankNodes(background);
}

private showFriendRank(): void {
    if (this.showOpenDataRank()) {
        return;
    }

    this.setBackgroundVisible(true);
}

private showOpenDataRank(): boolean {
    const rankManager = PlatformManager.getInstance();
    const canvas = rankManager.getOpenDataCanvas();
    if (!canvas) {
        return false;
    }

    this.setOpenDataCanvasSize(canvas);
    const postResult = rankManager.postOpenDataMessage({
        type: 'showFriendRank',
        key: RANK_KEY,
        width: RANK_VIEW_WIDTH,
        height: RANK_VIEW_HEIGHT,
    });
    if (postResult.result !== PlatformResult.Success) {
        return false;
    }

    this.m_RankSprite.node.active = true;

    this.startRefreshRankCanvas(canvas);
    return true;
}
```

```
private bindRankView(background: Node): void {
   const node = this.findChildByName(background, 'RankCanvasView') || this.createRankViewNode(background);

    const transform = node.getComponent(UITransform) || node.addComponent(UITransform);
    transform.setContentSize(RANK_VIEW_WIDTH, RANK_VIEW_HEIGHT);

    this.m_RankSprite = node.getComponent(Sprite) || node.addComponent(Sprite);
    this.m_RankSprite.sizeMode = Sprite.SizeMode.CUSTOM;
    node.active = false;
}

private hideUnusedPrefabRankNodes(background: Node): void {
    const scrollViewNode = this.findChildByName(background, 'ScrollView');
    if (scrollViewNode) scrollViewNode.active = false;

    const myRankItem = this.findChildByName(background, 'MyRankItem');
    if (myRankItem) myRankItem.active = false;
}

private createRankViewNode(parent: Node): Node {
    const node = new Node('RankCanvasView');
    node.layer = parent.layer;
    parent.addChild(node);
    node.setPosition(0, -40, 0);
    return node;
}

private startRefreshRankCanvas(canvas: unknown): void {
    this.stopRefreshRankCanvas();

    if (!canvas || !this.m_RankSprite) return;

    this.m_RankTexture = new Texture2D();
    (this.m_RankTexture as any).reset({
        width: RANK_VIEW_WIDTH,
        height: RANK_VIEW_HEIGHT,
    });

    const spriteFrame = new SpriteFrame();
    spriteFrame.texture = this.m_RankTexture;
    this.m_RankSprite.spriteFrame = spriteFrame;

    this.refreshRankCanvas(canvas);
    this.m_RefreshTimer = setInterval(() => this.refreshRankCanvas(canvas), 500);
    this.scheduleShowBackground(canvas);
}

private stopRefreshRankCanvas(): void {
    if (this.m_RefreshTimer) {
        clearInterval(this.m_RefreshTimer);
```

```
        this.m_RefreshTimer = null;
    }
    this.m_RankTexture = null;
}

private scheduleShowBackground(canvas: unknown): void {
    this.clearShowBackgroundTimer();
    this.m_ShowBackgroundTimer = setTimeout(() => {
        this.refreshRankCanvas(canvas);
        this.setBackgroundVisible(true);
        this.m_ShowBackgroundTimer = null;
    }, 800);
}

private clearShowBackgroundTimer(): void {
    if (this.m_ShowBackgroundTimer) {
        clearTimeout(this.m_ShowBackgroundTimer);
        this.m_ShowBackgroundTimer = null;
    }
}

private refreshRankCanvas(canvas: unknown): void {
    if (!this.m_RankTexture || !this.m_RankSprite || !this.m_RankSprite.isValid) return;

    const texture = this.m_RankTexture as any;
    if (texture.uploadData) {
        texture.uploadData(canvas);
    }
}

private setOpenDataCanvasSize(canvas: unknown): void {
    const openDataCanvas = canvas as { width?: number; height?: number };
    openDataCanvas.width = RANK_VIEW_WIDTH;
    openDataCanvas.height = RANK_VIEW_HEIGHT;
}

private setBackgroundVisible(visible: boolean): void {
    if (this.m_BackgroundNode && this.m_BackgroundNode.isValid) {
        this.m_BackgroundNode.active = visible;
    }
}

private createLabelNode(name: string, text: string, fontSize: number, x: number, y: number): Label {
    const node = new Node(name);
    node.layer = this.node.layer;
    this.node.addChild(node);
    node.setPosition(x, y, 0);

    const transform = node.addComponent(UITransform);
    transform.setContentSize(680, 60);
```

```
        const label = node.addComponent(Label);
        label.string = text;
        label.fontSize = fontSize;
        label.lineHeight = fontSize + 6;
        label.horizontalAlign = Label.HorizontalAlign.CENTER;
        label.verticalAlign = Label.VerticalAlign.CENTER;
        return label;
    }

    private closeRankPanel(): void {
        if (this.m_UIID) {
            this.CloseSelf();
            return;
        }

        this.OnClose();
        this.node.removeFromParent();
        this.node.destroy();
    }

    private getBackgroundNode(): Node {
        return this.findChildByName(this.node, 'Background') || this.node;
    }

    private findChildByName(root: Node, name: string): Node | null {
        if (!root) return null;
        if (root.name === name) return root;

        for (const child of root.children) {
            const matched = this.findChildByName(child, name);
            if (matched) return matched;
        }

        return null;
    }
}
import { _decorator, Node, tween, Vec3 } from 'cc';
import { UIBase } from '../../../engine/ui/UIBase';
import { UIManager } from '../../../engine/ui/UIManager';
import { CommonUIID } from '../CommonUIConfig';
const { ccclass } = _decorator;

export interface TransitionPanelOptions {
    onComplete?: () => void;
}

@ccclass('TransitionPanel')
export class TransitionPanel extends UIBase {
    private m_Root: Node = null;
```

```
    private m_OnComplete: (() => void) | null = null;

    OnOpen(options: TransitionPanelOptions = {}): void {
        this.m_OnComplete = options.onComplete || null;
        this.m_Root = this.findChildByName(this.node, 'Root') || this.node;
        this.playTransition();
    }

    OnClose(): void {
        if (this.m_Root && this.m_Root.isValid) {
            tween(this.m_Root).stop();
        }
        this.m_OnComplete = null;
        this.m_Root = null;
        super.OnClose();
    }

    private playTransition(): void {
        if (!this.m_Root || !this.m_Root.isValid) {
            this.finishTransition();
            return;
        }

        this.m_Root.setScale(new Vec3(0.85, 0.85, 1));
        tween(this.m_Root)
            .to(0.2, { scale: new Vec3(1.08, 1.08, 1) })
            .to(0.12, { scale: new Vec3(1, 1, 1) })
            .delay(0.25)
            .call(() => this.finishTransition())
            .start();
    }

    private finishTransition(): void {
        const onComplete = this.m_OnComplete;
        UIManager.GetInstance().ClosePanel(CommonUIID.TransitionPanel);
        onComplete?.();
    }

    private findChildByName(root: Node, name: string): Node | null {
        if (!root) return null;
        if (root.name === name) return root;

        for (const child of root.children) {
            const matched = this.findChildByName(child, name);
            if (matched) return matched;
        }

        return null;
    }
}
```